Worksheet 11 — Memory-Safety & Exploitation (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 11 **Aligned:** CWE-121 (stack-based buffer overflow) · CWE-787 (out-of-bounds write) · CWE-134 (externally-controlled format string) · CWE-242 (dangerous `gets`) **Signature game:** 💥 Fuzzing Race → Pwn the Binary
>
> **Ethics note:** The binaries built from `vuln.c` are **intentionally exploitable** and `win()` spawns `/bin/sh`. Build and run them **only inside the isolated toolbox container** (or a VM) — never on your host. Never run these on a machine you care about, and never aim these techniques at software you are not authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Supichaya Rabiabnaweenurak	6631503118	23/10/2569	---

Part 2 — Lecture Questions

1. Draw the x86-64 stack frame for `parse_input()` in `vuln.c` (buf[64], saved RBP, saved return address). Where does `strcpy` start writing and which word must the exploit overwrite?
2. Why is `gets()` (CWE-242) impossible to use safely, and why did C11 remove it from the standard library?
3. Explain CWE-134: why is `printf(user_input)` dangerous but `printf("%s", user_input)` safe? What can `%n` do?
4. Name the four mitigations in the hardened build (`make vuln-hardened`) — stack canary, PIE/ASLR, NX/DEP, FORTIFY — and say which one each defeats: ret2win, shellcode-on-stack, return-address overwrite detection.
5. The CISA/ONCD roadmaps push memory-safe languages. Using `safe.rs`, explain why CWE-121 is *impossible* in safe Rust, not merely harder.

Answers

1. On x86-64, the local array occupies the lowest part of the frame, followed by the saved base pointer and then the saved return address:

```
higher addresses
[saved return address]  <- 72 bytes from buf[0]
[saved RBP]             <- 64 bytes from buf[0]
[buf[64]]               <- strcpy starts writing here
lower addresses
```

`strcpy` starts at `buf[0]` and copies until the NUL terminator without checking the destination size. After 64 bytes of `buf` and 8 bytes of saved RBP, the exploit must overwrite the saved return-address word.

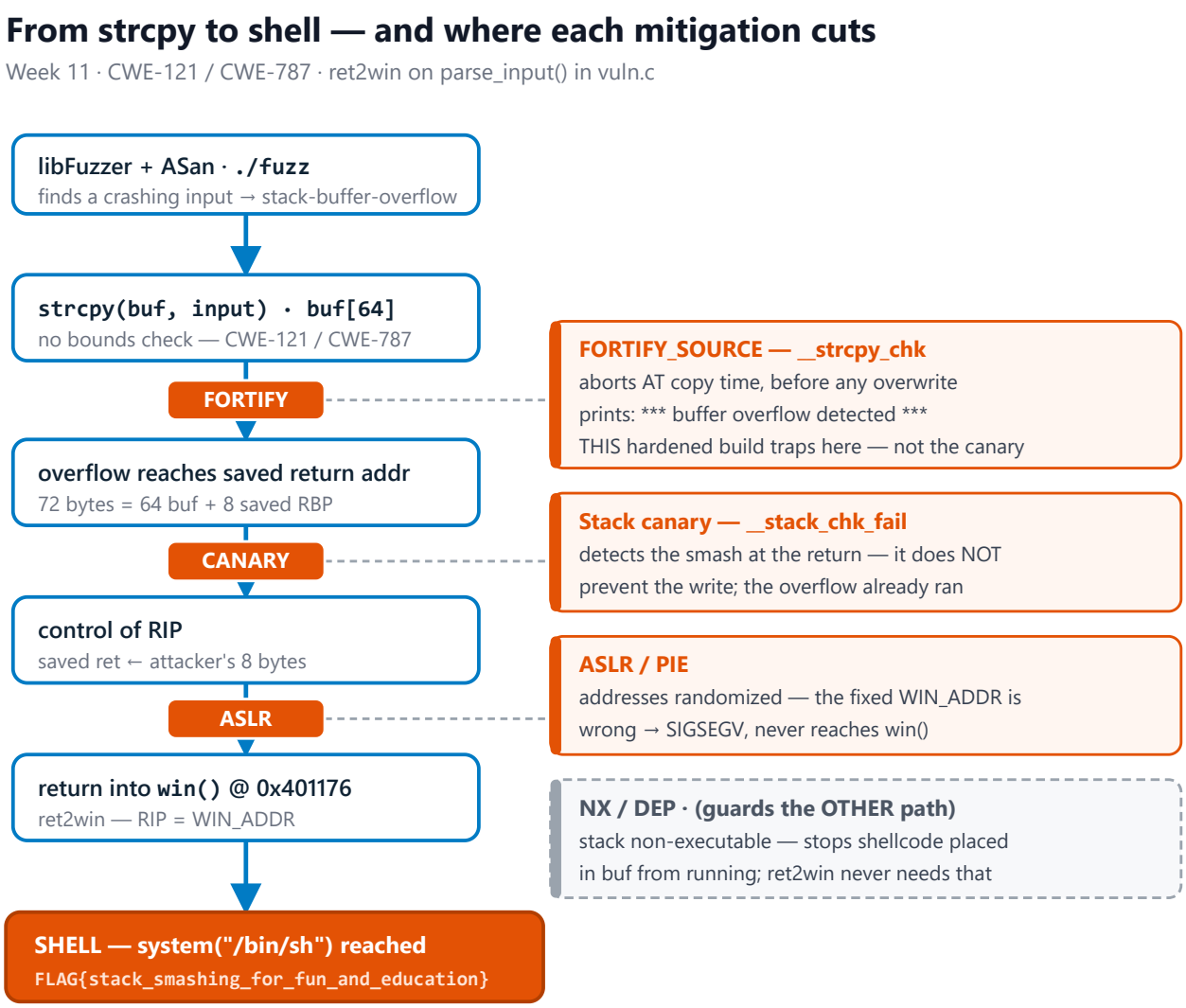
2. `gets()` accepts input until a newline or EOF but has no argument for the destination buffer's size, so an input longer than the buffer always risks overwriting adjacent memory. It was removed from the C11 standard because no caller can use it safely without an external input-length guarantee; safer alternatives include `fgets()` with an explicit buffer size.

3. In `printf(user_input)`, the input is interpreted as a format string, so sequences such as `%x` can disclose stack values and `%n` can write the number of characters printed to an attacker-chosen address. In `printf("%s", user_input)`, the format string is a fixed literal and the input is treated only as data, so format metacharacters are not interpreted.

4. **Stack canaries** detect a stack overwrite before the function returns and therefore defeat return-address overwrite attempts. **FORTIFY** adds object-size checks to functions such as `strcpy` and can stop the overflow before the canary check. **PIE/ASLR** randomizes code and stack addresses, so a hard-coded `win()` address for ret2win is unreliable. **NX/DEP** marks the stack non-executable, defeating shellcode placed on the stack; it does not by itself stop ret2win because ret2win reuses existing executable code.

5. In safe Rust, `String` owns a length-tracked allocation and `&str` carries a checked view into string data; safe indexing and copying operations perform bounds checks instead of writing past an array. The compiler also rejects unsafe format-string usage such as `println!(line)` because the format string must be a compile-time literal. Therefore safe Rust has no operation that can overwrite a C-style stack return address, making CWE-121 impossible in safe Rust code unless `unsafe` code or an unsafe FFI boundary is introduced.

Part 3 — Hands-on Lab (145 min)



Learning goals: Find the overflow with a coverage-guided fuzzer, exploit it (ret2win), watch mitigations break the exploit, then port the routine to safe Rust.

Prerequisites: the **toolbox container** (bundles `clang`+libFuzzer, `gdb`, `objdump`, `python3`) — Apple clang has no libFuzzer and `gdb` is painful on macOS, so do this week in the container. (`pip install pwntools` and `rustc` inside as needed; an isolated Linux VM also works.)

Environment setup

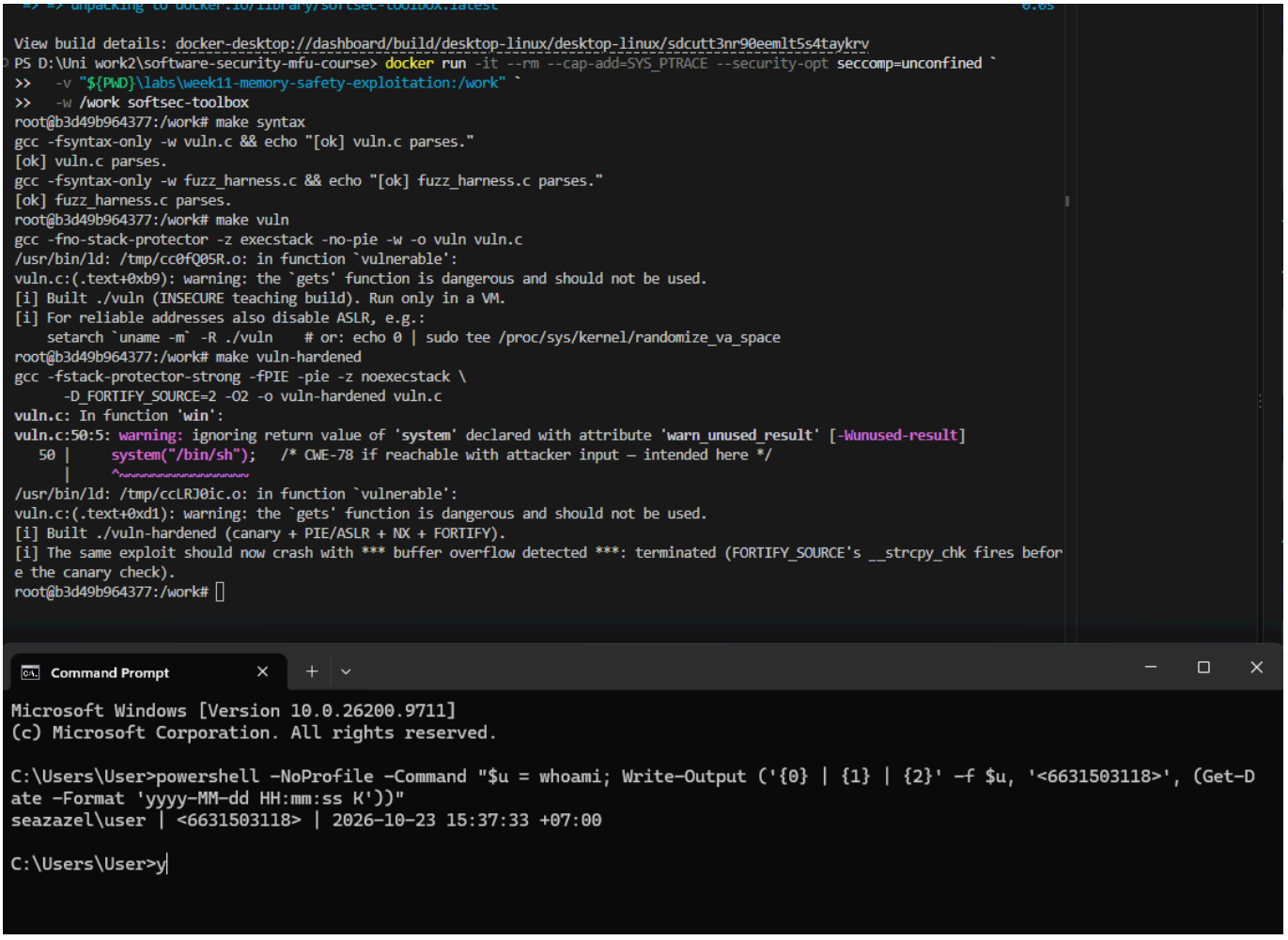
```
# from the repo root, build the toolbox once, then open a shell IN this week's
folder:
docker build -t softsec-toolbox labs/toolbox
docker run -it --rm --cap-add=SYS_PTRACE --security-opt seccomp=unconfined \
    -v "$PWD/labs/week11-memory-safety-exploitation":/work -w /work softsec-
toolbox
# --- inside the container ---
make vuln                                          # teaching build: mitigations
OFF
```

```
make vuln-hardened                                  # mitigations ON
clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz   # libFuzzer + ASan
setarch `uname -m` -R ./vuln                        # disable ASLR for repeatable
addresses
```

What to submit per task: the exact command(s) + crash/exploit output, a screenshot, and a 2–3 sentence mitigation note tied to the CWE.

Task 0 — Onboarding (15 min)

Run `make syntax` (parses both C files even on macOS) and confirm `make vuln` / `make vuln-hardened` build. Read the bug-class banner in `vuln.c`. **Deliverable:** build output + the list of CWEs `vuln.c` claims to demonstrate.



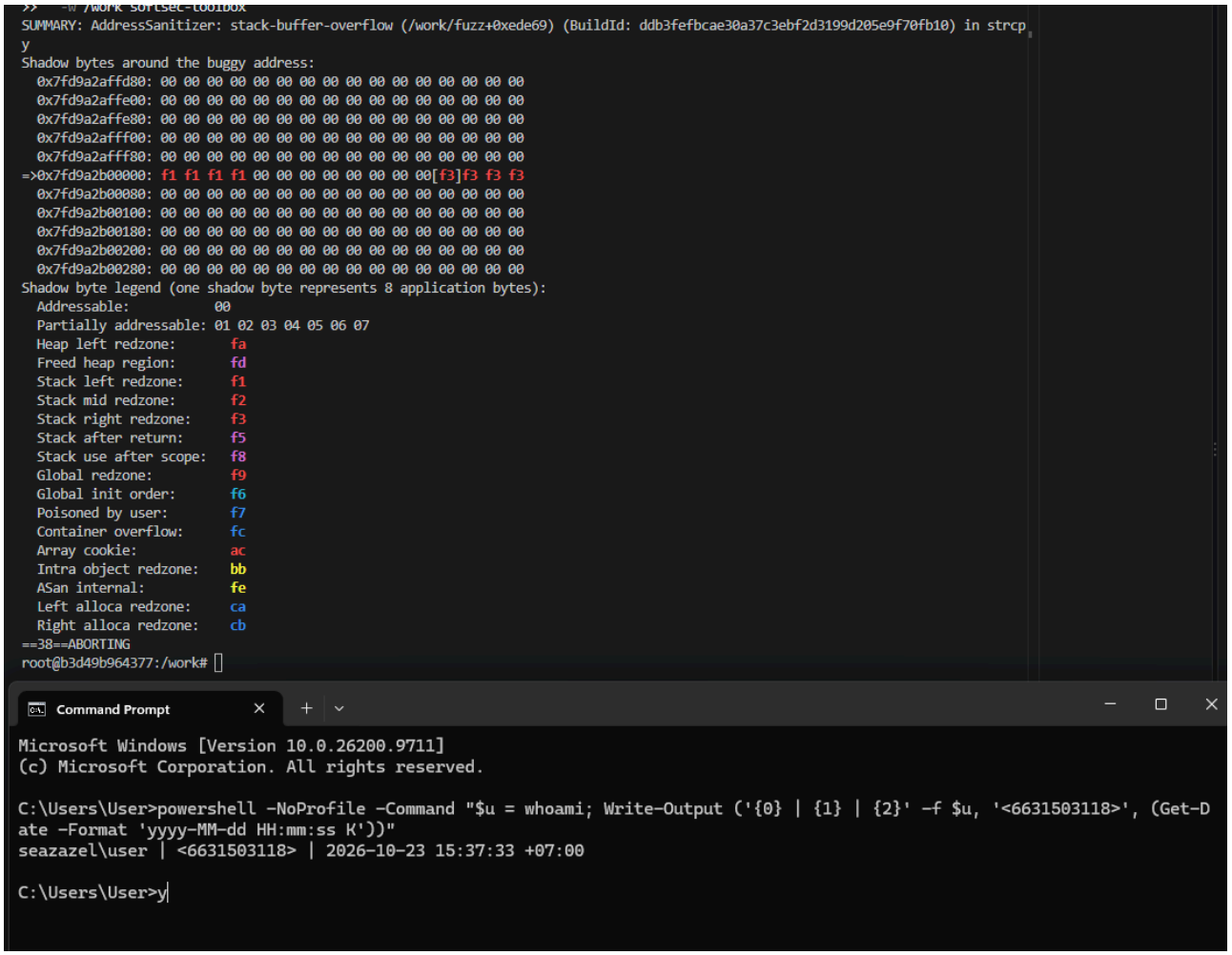
> - CWE-121, CWE-787, CWE-134, CWE-242

Task 1 — Fuzzing Race: find the crash (35 min)

Goal: Make libFuzzer + AddressSanitizer crash on the `strcpy` overflow in `parse_input()` — CWE-121/787.
Steps:

1. `./fuzz -runs=100000` (or just `./fuzz`).
2. Read the ASan report — note `stack-buffer-overflow` inside `parse_input` and the saved `crash-<hash>` file.

3. Replay it: `./fuzz crash-<hash>`. **Deliverable:** the ASan crash summary + the byte length of the crashing input + mitigation note.



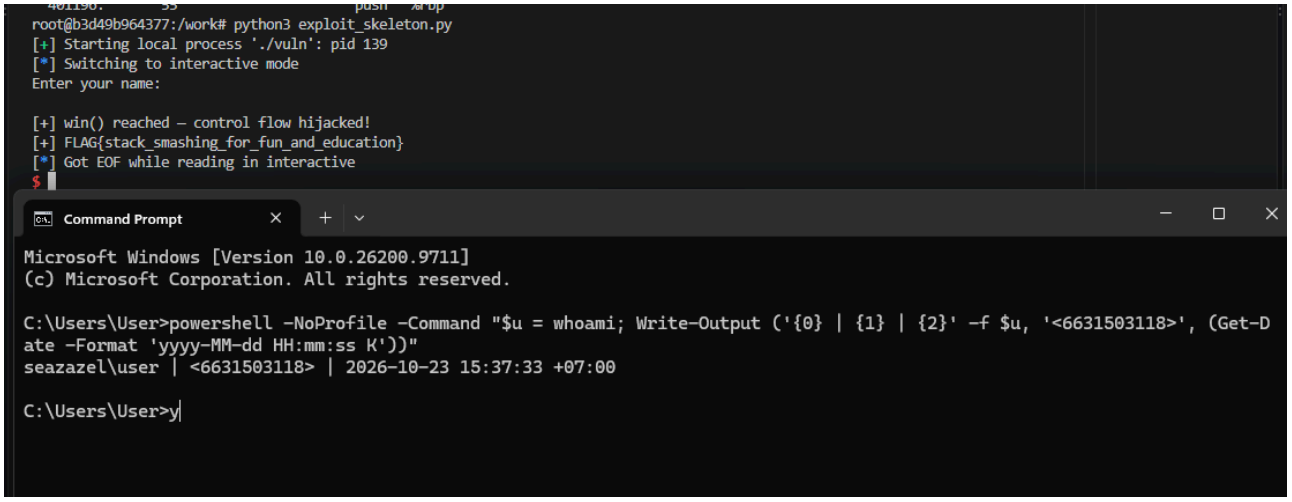
> - strcpy() writes beyond buf[64], causing CWE-121/CWE-787. Use bounded input functions and validate lengths before copying.

Task 2 — Pwn the Binary: ret2win (40 min)

Goal: Overwrite the saved return address so the binary returns into `win()` and prints `FLAG{stack_smashing_for_fun_and_education}` — CWE-121. **Steps:**

1. Find `win()`'s address: `objdump -d vuln | grep '<win>:'` (or `gdb -batch -ex 'p win' ./vuln`).
2. Verify the offset with a cyclic pattern (do **not** trust 72 blindly): feed `cyclic(200)`, crash in gdb, `cyclic_find` the value in RIP. (On an Apple Silicon/arm64 Mac the toolbox has no platform pin, so it builds as an **aarch64** container — there is no `RIP`; use `info registers pc` / `p/x $pc` instead, and expect `SIGBUS`, not `SIGSEGV`. The offset itself is still 72 here.)
3. Edit `exploit_skeleton.py`: set `WIN_ADDR` to the address from step 1, confirm `OFFSET`.
4. `python3 exploit_skeleton.py` (pwntools path) — or `python3 exploit_skeleton.py | ./vuln`. (On arm64: the flag still prints, but the process then loops forever re-printing `[+] win() reached` instead of exiting — ARM64 keeps the return address in register `x30`, which still holds `win()`'s own address when it starts, so `win()`'s `ret` returns into itself. Expected; `Ctrl-C` once you've captured the

flag.) **Deliverable:** the verified `OFFSET`, your `WIN_ADDR`, and the printed flag / `[+] win() reached` line.

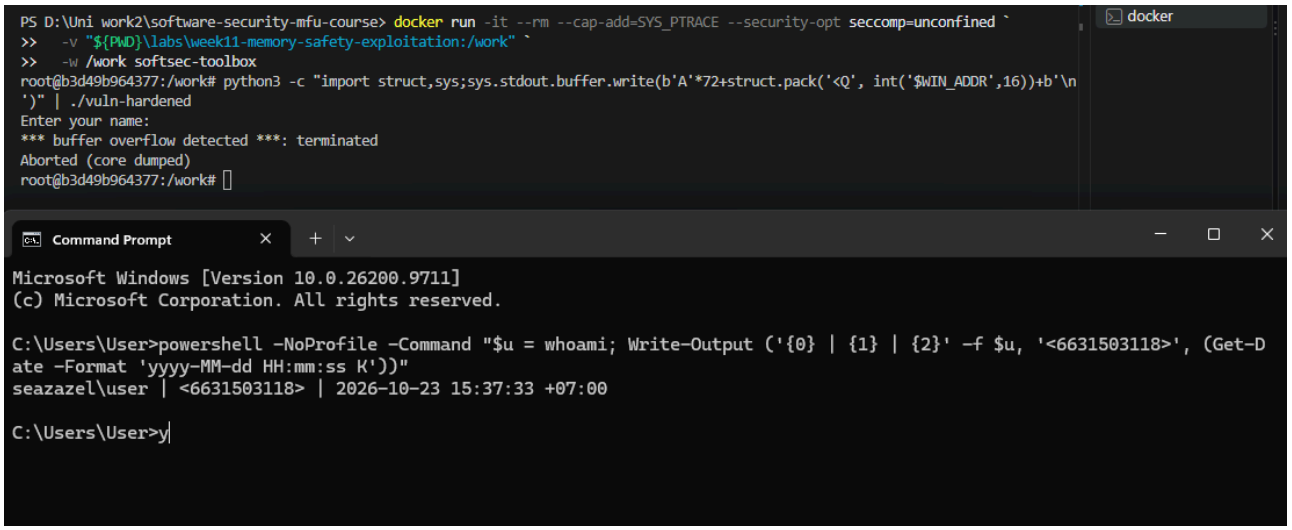


Task 3 — Hardened rebuild: watch the exploit break (25 min)

```
stack-frame
```

Goal: Observe modern mitigations defeating the same payload. **Steps:**

1. Run your Task 2 payload against `./vuln-hardened`.
2. Record the failure mode: `*** buffer overflow detected ***: terminated` (FORTIFY_SOURCE's `__strcpy_chk`, which on this build catches the overflow before the stack canary's epilogue check ever runs), or a SIGSEGV at a randomized address (PIE/ASLR makes `WIN_ADDR` wrong). **Deliverable:** the crash message + a note explaining *which* mitigation fired and why the fixed `WIN_ADDR` no longer works.



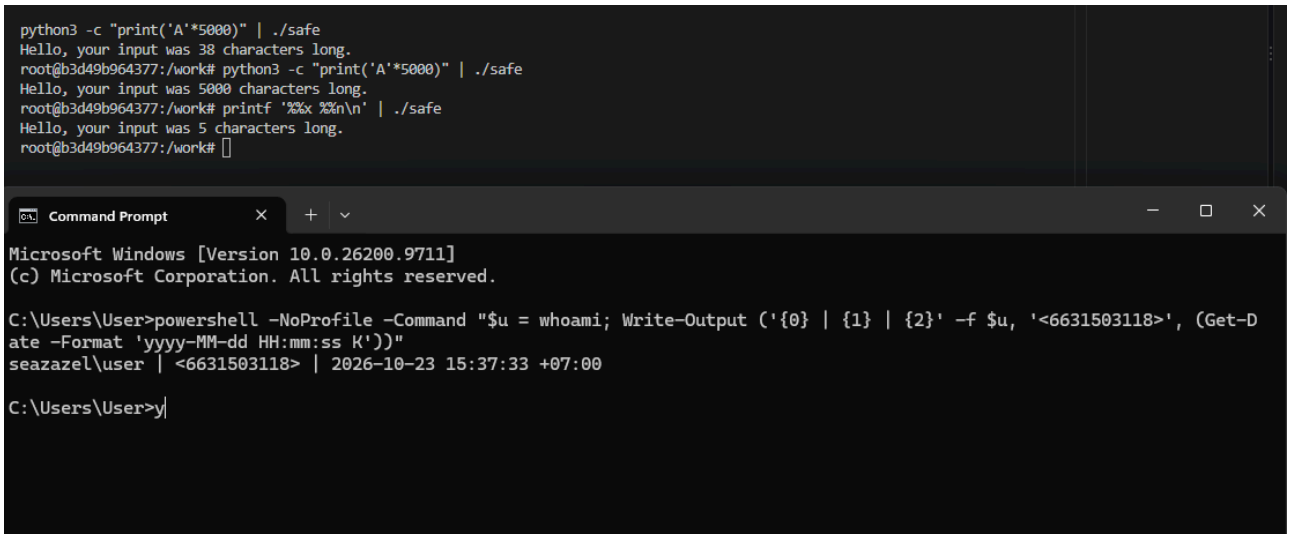
> -This is FORTIFY detecting the overflowing strcpy() before the stack canary check. PIE/ASLR also makes the old fixed win() address unreliable.

Task 4 — Rust rewrite: defend / fix it (30 min)

Goal: Show the bug class disappears in safe Rust using `safe.rs`. **Steps:**

1. `rustc safe.rs && ./safe`, then `python3 -c "print('A'*5000)" | ./safe` — note it never overflows or crashes.

2. Feed format-string metacharacters: `printf '%%x %%n' | ./safe` — note the program just reports the character count with no crash and no stack leak; the input is treated as ordinary data (CWE-134 impossible: `println!`'s format string must be a compile-time literal, so `println!(line)` would not even compile).
3. From the inline comments, write *why* `String`/`&str` + bounds checks make CWE-121/787 impossible. **Deliverable:** the safe output for both inputs + a 3–4 sentence explanation tying it to the CISA memory-safe roadmap.]



> -Rust uses length-tracked String and &str values instead of unchecked C arrays and strcpy(). Bounds checks, ownership, and the borrow checker prevent out-of-bounds writes in safe Rust. Rust format strings must be compile-time literals, so user input cannot become a format string.

Part 4 — Reflection

1. **Mapping:** table — bug | function in `vuln.c` | CWE | mitigation that stops it (canary / PIE / NX / Rust).
2. **Real breach:** pick a memory-safety incident (Morris Worm fingerd overflow, or Heartbleed CVE-2014-0160). Which CWE applies, and would a stack canary or a Rust rewrite have prevented it?
3. **Best mitigation:** compare "harden the C build" vs. "rewrite in Rust." Which gives durable risk reduction for new code, and what is the cost?

Reflection answers

Bug	Function in `vuln.c`	CWE	Mitigation that stops it
Stack-based buffer overflow	`parse_input()` / `strcpy(buf, src)`	CWE-121, CWE-787	Bounds-checked copying, FORTIFY, stack canary, or safe Rust
Dangerous unbounded input	`vulnerable()` / `gets(line)`	CWE-242	Replace `gets()` with bounded `fgets()` and validate the length
Externally controlled format string	`format_bug()` / `printf(user_input)`	CWE-134	Use a literal format string such as `printf("%s", user_input)`

The Morris Worm's attack against the Unix `fingerd` service included a stack buffer overflow, which maps to CWE-121. A compiler-inserted stack canary could have detected many return-address overwrites in a hardened build, but it is a mitigation rather than a complete repair. Rewriting the vulnerable routine in safe

Rust would prevent this class of out-of-bounds stack write, provided the implementation did not use `unsafe` code or an unsafe FFI boundary.

For new code, a safe-Rust rewrite gives more durable risk reduction because the compiler and runtime checks prevent entire classes of memory-corruption bugs rather than only detecting selected overwrite patterns. Hardening a C build is still valuable for existing code and is cheaper and more incremental, but it can be bypassed or missed by a different memory-safety bug. Rust's costs include migration effort, retraining, interoperability with C libraries, and possible performance or tooling changes.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–3: fuzz crash, ret2win flag, hardened break)	40
Defense (Task 4: Rust rewrite + explanation)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
    1. What did you do, and **why did the vulnerability work**?
    2. **Why does your fix actually stop it** — and what could still break it?

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Audit response

The AI guidance said to use a placeholder command such as `python3 -c "... struct.pack('<Q',0x401176) ..." | ./vuln`. That is risky because `0x401176` is only an example address; in this build `objdump` reported `win()` at `0x401196`, while `0x401229` pointed into `parse_input()`. Using the wrong address caused the saved `RIP` to jump into the wrong instruction and produced `SIGSEGV`.

The corrected process is to obtain the address from the actual binary with `objdump -d vuln | grep -A1 '<win>:'`, verify the 72-byte offset, set `WIN_ADDR = 0x401196`, and then rerun the exploit inside the toolbox container. The earlier fallback advice also needed care because the skeleton prints its pwntools warning to standard output when pwntools is absent; that warning can corrupt a payload when stdout is piped directly into `vuln`.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Comprehension answers

A. Explain in Plain English: The C program copies an input of any length into a 64-byte stack buffer with `strcpy`, so a long input overwrites the saved frame pointer and return address. By replacing that return address with the address of `win()`, the input changes the program's control flow and reaches code that was never called by `main()`.

B. Final prompt:

> Rewrite the `parse_input` behavior from this Week 11 C lab in safe Rust. Read all stdin into a growable `String`, report its character count, and print user data only through a literal format string. Do not use `unsafe`, raw pointers, fixed-size unchecked copies, `strcpy`, or runtime format strings. Explain why the resulting code prevents CWE-121, CWE-787, and CWE-134, and provide commands to test a 5,000-character input and the literal text `%x %n`.

Verified result: Compiling with `rustc safe.rs -o safe` and piping 5,000 `A` characters produced `Hello, your input was 5000 characters long.` without a crash. Piping `%x %n` produced a normal character-count message; the metacharacters were treated as input data, not as a format string.